



HINDUSTAN

INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF COMPUTER APPLICATIONS

BACHELOR OF COMPUTER APPLICATIONS

(SEMESTER – II)

ACA31006 – Data Base Management Systems

Integrated Lab Record

Name : _____

Register No : _____



HINDUSTAN

INSTITUTE OF TECHNOLOGY & SCIENCE
(DEEMED TO BE UNIVERSITY)

DEPARTMENT OF COMPUTER APPLICATIONS

BACHELOR OF COMPUTER APPLICATIONS
(SEMESTER – II)

Bonafide Certificate

This is a certified bonafide record of work done by
_____ (Register Number _____)
of _____ class, submitted for the End Semester Practical Examination
held on _____ in **ACA31006 – DATA BASE MANAGEMENT
SYSTEMS** Laboratory during the academic year **2024 – 2025**.

Staff In-Charge

Internal Examiner

External Examiner

Index				
S.No	Date	Title	Page Number	Faculty's Signature
1.		Library Management System using DDL Commends		
2.		Vehicle Parking Management System using DDL and DML Commands		
3.		Student Score Card Preparation and draw an ER Diagram		
4.		Supermarket System and draw an ER Diagram		
5.		Computer Goods table for Purchase and Sales to perform relational algebra		
6.		Bank Customer table with Primary Key and execute 1NF and 2NF		
7.		Salary table and apply Statistical functions		
8.		Explore about single row function – Date functions		
9.		Explore about single row function – String functions		
10.		Display a hash value using default hash algorithm		

	Rubric's of the Lab Practical's					
S.No	Title	Aim (2)	Procedure (3)	Code (3)	Output (2)	Total (10)
1.	Library Management System using DDL Commends					
2.	Vehicle Parking Management System using DDL and DML Commands					
3.	Student Score Card Preparation and draw an ER Diagram					
4.	Supermarket System and draw an ER Diagram					
5.	Computer Goods table for Purchase and Sales to perform relational algebra					
6.	Bank Customer table with Primary Key and execute 1NF and 2NF					
7.	Salary table and apply Statistical functions					
8.	Explore about single row function – Date functions					
9.	Explore about single row function – String functions					
10.	Display a hash value using default hash algorithm					

Aim:

To create the table for library management system using DDL (Data Definition Language) commands in SQL Plus platform.

Procedure:

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Create the table for library management system
- ❖ **Step 3:** The attributes are Book_no, Book_name, Auther_Name, Publisher_name, Issued_Date, Received_Date,
- ❖ **Step 4:** Execute the DDL commends for given table
- ❖ **Step 5:** Stop

The following table describes about DDL commands

DDL Commands	Description
Create	to create a table
Alter	to alter the structure of the table
Drop	to delete objects from the table
Rename	to rename an object existing in the table

The following table expressed about attributes for library table

Attributes	Description	Datatype
Book_no	Book Number	Integer
Book_name	Book Name	Varchar(20)
Auther_Name	Auther Name	Varchar(25)
Publisher_name	Publisher Name	Varchhar(30)
Issued_Date	Issued Date	Date
Received_Date	Received Date	Date

Structured Query Language (SQL)

Create Query:

```
SQL> create table library_sys(Book_no int,  
2   Book_name varchar(30),  
3   Auther_Name Varchar(30),  
4   Publisher_name varchar(30),  
5   Issued_Date date,  
6   Received_Date date  
7 );
```

Table created.

```
SQL> desc library_sys;
```

Name	Null?	Type
BOOK_NO		NUMBER(38)
BOOK_NAME		VARCHAR2(30)
AUTHER_NAME		VARCHAR2(30)
PUBLISHER_NAME		VARCHAR2(30)
ISSUED_DATE		DATE
RECEIVED_DATE		DATE

Alter-Add Column Query:

```
SQL> Alter table library_sys add Due_amt int;
```

Table altered.

```
SQL> desc library_sys;
```

Name	Null?	Type
BOOK_NO		NUMBER(38)
BOOK_NAME		VARCHAR2(30)
AUTHER_NAME		VARCHAR2(30)
PUBLISHER_NAME		VARCHAR2(30)
ISSUED_DATE		DATE
RECEIVED_DATE		DATE
DUE_AMT		NUMBER(38)

Alter-Drop Column Query:

SQL> Alter table library_sys drop column Issued_Date;

Table altered.

SQL> desc library_sys;

Name	Null?	Type

BOOK_NO		NUMBER(38)
BOOK_NAME		VARCHAR2(30)
AUTHER_NAME		VARCHAR2(30)
PUBLISHER_NAME		VARCHAR2(30)
RECEIVED_DATE		DATE
DUE_AMT		NUMBER(38)

Rename - Table Name Query:

SQL> Alter table library_sys rename to lib;

Table altered.

SQL> desc library_sys;

ERROR: ORA-04043: object library_sys does not exist

SQL> desc lib;

Name	Null?	Type

BOOK_NO		NUMBER(38)
BOOK_NAME		VARCHAR2(30)
AUTHER_NAME		VARCHAR2(30)
PUBLISHER_NAME		VARCHAR2(30)
RECEIVED_DATE		DATE
DUE_AMT		NUMBER(38)

Rename - Column Name Query:

SQL> alter table lib rename column Book_no to S_no;

Table altered.

SQL> desc lib;

Name	Null?	Type
S_NO		NUMBER(38)
BOOK_NAME		VARCHAR2(30)
AUTHER_NAME		VARCHAR2(30)
PUBLISHER_NAME		VARCHAR2(30)
RECEIVED_DATE		DATE
DUE_AMT		NUMBER(38)

Drop – Table Query:

SQL> Drop table lib;

Table Dropped

Result:

Thus, the above table was created and executed successfully for library management system.

Ex No: 2	Vehicle Parking System Using DDL and DML Commands

Aim

To create the table for vehicle parking management system using DDL (Data Definition Language) and DML (Data Manipulation Language) commands.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Create the table for vehicle parking system
- ❖ **Step 3:** The attributes are vech_no, vech_type, prk_date, in_ti, ot_ti, slt_no and prk_amt
- ❖ **Step 4:** Execute the DDL and DML commands for the given table
- ❖ **Step 5:** Stop

The following table describes about DDL commands

Commands	Description
Create	to create a table
Alter	to alter the structure of the table
Drop	to delete objects from the table
Rename	to rename an object existing in the table

The following table describes about DML commands

Commands	Description
Select	to retrieve data from one or more tables
Insert	to add new records in a table
Delete	to delete existing records from a table.
Update	to modify existing records in a table

The following table expressed about attributes for vehicle parking system table

Attributes	Description	Datatype
vech_no	Vehicle Number	Int
vech_type	Type of Vehicle	Varchar(30)
prk_date	Parking Date	Date
in_ti	Parking In-Time	Date
ot_ti	Parking Out-Time	Date
slt_no	Allocated Place Number	Int
prk_amt	Parking Amount	Int

Structured Query Language (SQL)

```
SQL> create table vehicle_sys(vech_no int,
2  vech_type varchar(30),
3  prk_dt date,
4  in_ti varchar2(30),
5  ot_ti varchar2(30),
6  slt_no int,
7  prk_amt int);
```

Table created.

```
SQL> desc vehicle_sys;
```

Name	Null?	Type
VECH_NO		NUMBER(38)
VECH_TYPE		VARCHAR2(30)
PRK_DT		DATE
IN_TI		VARCHAR2(30)
OT_TI		VARCHAR2(30)
SLT_NO		NUMBER(38)
PRK_AMT		NUMBER(38)

```
SQL> insert into vehicle_sys values(0120,'Two-Whl','25/april/24','10.30 am','4.35 pm',203,20);
```

1 row created.

```
SQL> insert into vehicle_sys values(0151,'Two-Whl','20-Mar-24','11.30 am','3.15 pm',120,20);
```

1 row created.

```
SQL> insert into vehicle_sys values(0001,'Four-Whl','21-Feb-24','8.40 am','5.35 pm',290,50);
```

1 row created.

```
SQL> insert into vehicle_sys values(2546,'Two-Whl','2-Jan-24','9.20 am','8.05 pm',15,20);
```

1 row created.

```
SQL> insert into vehicle_sys values(3654,'Four-Whl','5/10/2023','5.00 pm','10.35 pm',26,50);
```

1 row created.

```
SQL> insert into vehicle_sys values(3129,'Two-Whl','1/15/2024','8.00 am','11.00 pm',189,20);
```

1 row created.

```
SQL> insert into vehicle_sys values(6039,'Four-Whl','8-Dec-23','3.00 pm','9.20 pm',145,50);
```

1 row created.

```
SQL> insert into vehicle_sys values(4378,'Four-Whl','3-Nov-23','2.00 pm','4.40 pm',67,50);
```

1 row created.

```
SQL> insert into vehicle_sys values(8089,'Four-Whl','5-Oct-23','10.30 am','2.20 pm',200,50);
```

1 row created.

```
SQL> insert into vehicle_sys values(2156,'Two-Whl','25/april/24','7.45 am','1.15 pm',100,20);
```

1 row created.

```
SQL> set lines 1000;
```

```
SQL> select * from vehicle_sys;
```

VECH_NO	VECH_TYPE	PRK_DT	IN_TI	OT_TI	SLT_NO	PRK_AMT
120	Two-Whl	25-Apr-24	10.3 am	4.35 pm	203	20
151	Two-Whl	20-Mar-24	11.3 am	3.15 pm	120	20
1	Four-Whl	21-Feb-24	8.4 am	5.35 pm	290	50
2546	Two-Whl	2-Jan-24	9.2 am	8.05 pm	15	20
3654	Four-Whl	10-May-23	5:00 PM	10.35 pm	26	50
3129	Two-Whl	15-Jan-24	8:00 AM	11:00 PM	189	20
6039	Four-Whl	8-Dec-23	3:00 PM	9.2 pm	145	50
4378	Four-Whl	3-Nov-23	2:00 PM	4.4 pm	67	50
8089	Four-Whl	5-Oct-23	10.3 am	2.2 pm	200	50
2156	Two-Whl	25-Apr-24	7.45 am	1.15 pm	100	20

10 rows selected.

Result:

Thus, the above table was created and executed successfully for vehicle parking management system.

Ex.No: 3	Student Score Card Preparation and draw an ER Diagram

Aim

To create student score card table and draw an Entity Relationship (ER) diagram.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Create the table for student score card system
- ❖ **Step 3:** The attributes are roll_no, name, course, sub1, sub2, sub3, sub4, sub5, tot, avg and result
- ❖ **Step 4:** Execute the SQL commends for given table
- ❖ **Step 5:** Stop

The following table expressed about attributes for student score card table

Attributes	Description	Datatype
Roll_No	Roll Number	Varchar(30)
Name	Name	Varchar(40)
Course	Course	Varchar(20)
Sub1	Subject 1	int
Sub2	Subject 2	int
Sub3	Subject 3	int
Sub4	Subject 4	int
Sub5	Subject 5	int
Tot	Total	int
Avg	Average	int
Result	Result	Varchar(10)

Structured Query Language (SQL)

```
SQL> create table stu_score(roll_no int,  
2 name varchar(25),  
3 course varchar(20),  
4 sub1 int,  
5 sub2 int,  
6 sub3 int,  
7 sub4 int,  
8 sub5 int,  
9 tot int,  
10 avg int,  
11 result varchar(20));
```

Table created.

```
SQL> desc stu_score;
```

Name	Null?	Type

ROLL_NO		NUMBER(38)
NAME		VARCHAR2(25)
COURSE		VARCHAR2(20)
SUB1		NUMBER(38)
SUB2		NUMBER(38)
SUB3		NUMBER(38)
SUB4		NUMBER(38)
SUB5		NUMBER(38)
TOT		NUMBER(38)
AVG		NUMBER(38)
RESULT		VARCHAR2(20)

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0120,'Babu','BBA',56,67,78,89,82);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0121,'Balu','CDF',60,40,56,78,50);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0122,'Bajaj','BCA',20,56,67,78,89);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0123,'kavin','MBA',60,59,69,40,56);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0124,'Merlin','MBA',50,67,89,45,60);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0125,'Ram','BCOM',89,56,78,80,56);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0126,'Parthi','BSC',50,67,79,89,40);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0127,'kabil','MSC',89,40,50,60,67);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0128,'Khaja','MCA',89,90,78,67,89);
```

1 row created.

```
SQL>          insert          into          stu_score(roll_no,name,course,sub1,sub2,sub3,sub4,sub5)
values(0129,'Chan','BCA',90,78,89,67,78);
```

1 row created.

SQL> set lines 300

SQL> set trimout on

SQL> set tab off

SQL> select * from stu_score;

ROLL_NO	NAME	COURSE	SUB1	SUB2	SUB3	SUB4	SUB5	TOT	AVG	RESULT
34	Anu	BCA	89	56	78	85	90			
120	Babu	BBA	56	67	78	89	82			
121	Balu	CDF	60	40	56	78	50			
122	Bajaj	BCA	20	56	67	78	89			
123	kavin	MBA	60	59	69	40	56			
124	Merlin	MBA	50	67	89	45	60			
125	Ram	BCOM	89	56	78	80	56			
126	Parthi	BSC	50	67	79	89	40			
127	kabil	MSC	89	40	50	60	67			
128	Khaja	MCA	89	90	78	67	89			
129	Chan	BCA	90	78	89	67	78			

11 rows selected.

SQL> update stu_score set tot=sub1+ sub2+ sub3+ sub4+ sub5;

11 rows updated.

SQL> select * from stu_score;

ROLL_NO	NAME	COURSE	SUB1	SUB2	SUB3	SUB4	SUB5	TOT	AVG	RESULT
34	Anu	BCA	89	56	78	85	90	398		
120	Babu	BBA	56	67	78	89	82	372		
121	Balu	CDF	60	40	56	78	50	284		
122	Bajaj	BCA	20	56	67	78	89	310		
123	kavin	MBA	60	59	69	40	56	284		
124	Merlin	MBA	50	67	89	45	60	311		
125	Ram	BCOM	89	56	78	80	56	359		
126	Parthi	BSC	50	67	79	89	40	325		
127	kabil	MSC	89	40	50	60	67	306		
128	Khaja	MCA	89	90	78	67	89	413		
129	Chan	BCA	90	78	89	67	78	402		

11 rows selected.

SQL> update stu_score set avg=tot/5;

11 rows updated.

SQL> select * from stu_score;

ROLL_NO	NAME	COURSE	SUB1	SUB2	SUB3	SUB4	SUB5	TOT	AVG	RESULT
34	Anu	BCA	89	56	78	85	90	398	80	
120	Babu	BBA	56	67	78	89	82	372	74	
121	Balu	CDF	60	40	56	78	50	284	57	
122	Bajaj	BCA	20	56	67	78	89	310	62	
123	kavin	MBA	60	59	69	40	56	284	57	
124	Merlin	MBA	50	67	89	45	60	311	62	
125	Ram	BCOM	89	56	78	80	56	359	72	
126	Parthi	BSC	50	67	79	89	40	325	65	
127	kabil	MSC	89	40	50	60	67	306	61	
128	Khaja	MCA	89	90	78	67	89	413	83	
129	Chan	BCA	90	78	89	67	78	402	80	

11 rows selected.

SQL> UPDATE stu_score

2 SET result =

3 CASE WHEN

4 Sub1>=40 and

5 Sub2 >=40 and

6 Sub3 >=40 and

7 Sub4 >=40 and

8 Sub5 >=40

9 THEN 'Pass' ELSE 'Fail' END;

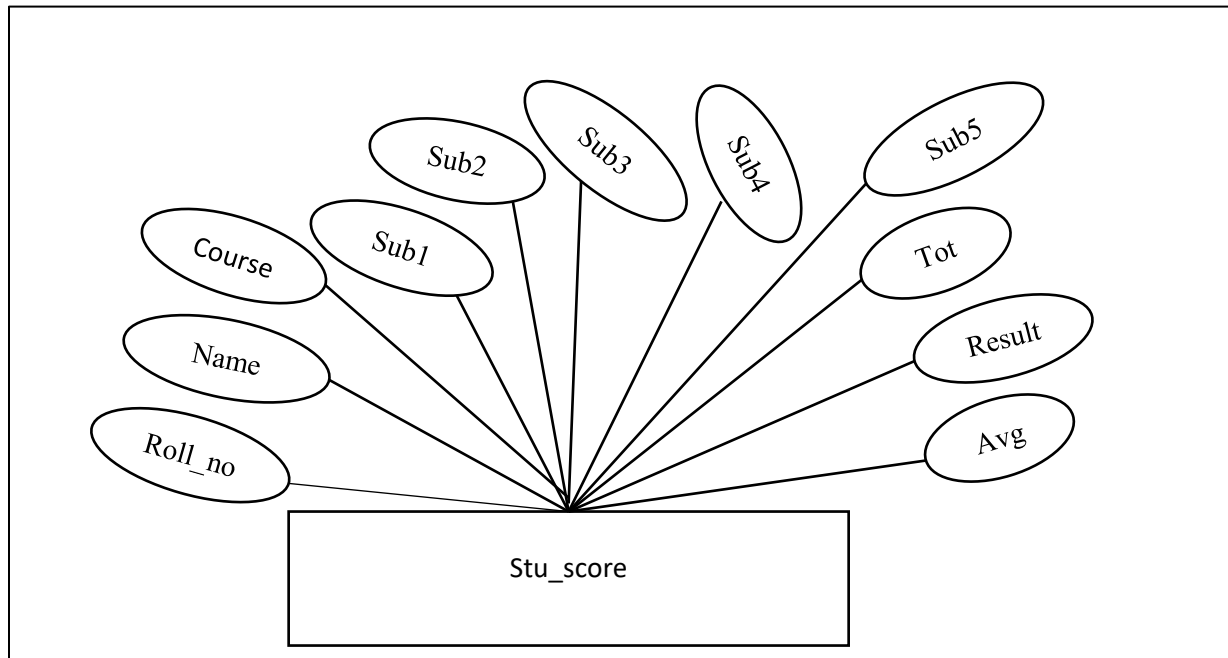
11 rows updated.

SQL> select * from stu_score;

ROLL_NO	NAME	COURSE	SUB1	SUB2	SUB3	SUB4	SUB5	TOT	AVG	RESULT
34	Anu	BCA	89	56	78	85	90	398	80	Pass
120	Babu	BBA	56	67	78	89	82	372	74	Pass
121	Balu	CDF	60	40	56	78	50	284	57	Pass
122	Bajaj	BCA	20	56	67	78	89	310	62	Fail
123	kavin	MBA	60	59	69	40	56	284	57	Pass
124	Merlin	MBA	50	67	89	45	60	311	62	Pass
125	Ram	BCOM	89	56	78	80	56	359	72	Pass
126	Parthi	BSC	50	67	79	89	40	325	65	Pass
127	kabil	MSC	89	40	50	60	67	306	61	Pass
128	Khaja	MCA	89	90	78	67	89	413	83	Pass
129	Chan	BCA	90	78	89	67	78	402	80	Pass

11 rows selected.

E-R Diagram



Result:

Thus, the student score card table was created and executed successfully.

Ex.No: 4	Supermarket System and draw an ER Diagram

Aim

To create supermarket table and draw an Entity Relationship (ER) diagram. **Procedure**

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Create the table for supermarket system
- ❖ **Step 3:** The attributes are bill_no,it_no,it_name,qty,dis,amt,tot_amt and gift
- ❖ **Step 4:** Execute the queries for given table
- ❖ **Step 5:** Stop

The following table expressed about attributes for supermarket system

Attributes	Description	Datatype
bill_no	Bill Number	Int
it_no	Item Number	Varchar(30)
it_name	Item Name	Varchar(30)
qty	Quantity	Int
dis	Discount	Int
amt	Amount	Int
tot_amt	Total Amount	Int
gift	Gift	Varchar(30)

Structured Query Language (SQL)

```
SQL> create table sup_mark_sys(bill_no int,  
  2 it_no varchar(30),  
  3 it_name varchar(30),  
  4 qty int,  
  5 dis int,  
  6 amt int,  
  7 tot_amt int,  
  8 gift varchar(10));
```

Table created.

```
SQL> desc sup_mark_sys;
```

Name	Null?	Type

BILL_NO		NUMBER(38)
IT_NO		VARCHAR2(30)
IT_NAME		VARCHAR2(30)
QTY		NUMBER(38)
DIS		NUMBER(38)
AMT		NUMBER(38)
TOT_AMT		NUMBER(38)
GIFT		VARCHAR2(10)

```
SQL> insert into sup_mark_sys values(10,'veg_30','Potato',30,0,100,(100-0),'N');
```

1 row created.

```
SQL> insert into sup_mark_sys values(10,'veg_10','Onion',50,20,400,(400-20),'Y');
```

1 row created.

```
SQL> insert into sup_mark_sys values(11,'frts_2','Papaya',5,20,300,(300-20),'Y');
```

1 row created.

```
SQL> insert into sup_mark_sys values(10,'veg_10','Onion',50,20,400,(400-20),'Y');
```

1 row created.

```
SQL> insert into sup_mark_sys values(11,'frts_2','Papaya',5,20,300,(300-20),'Y');
```

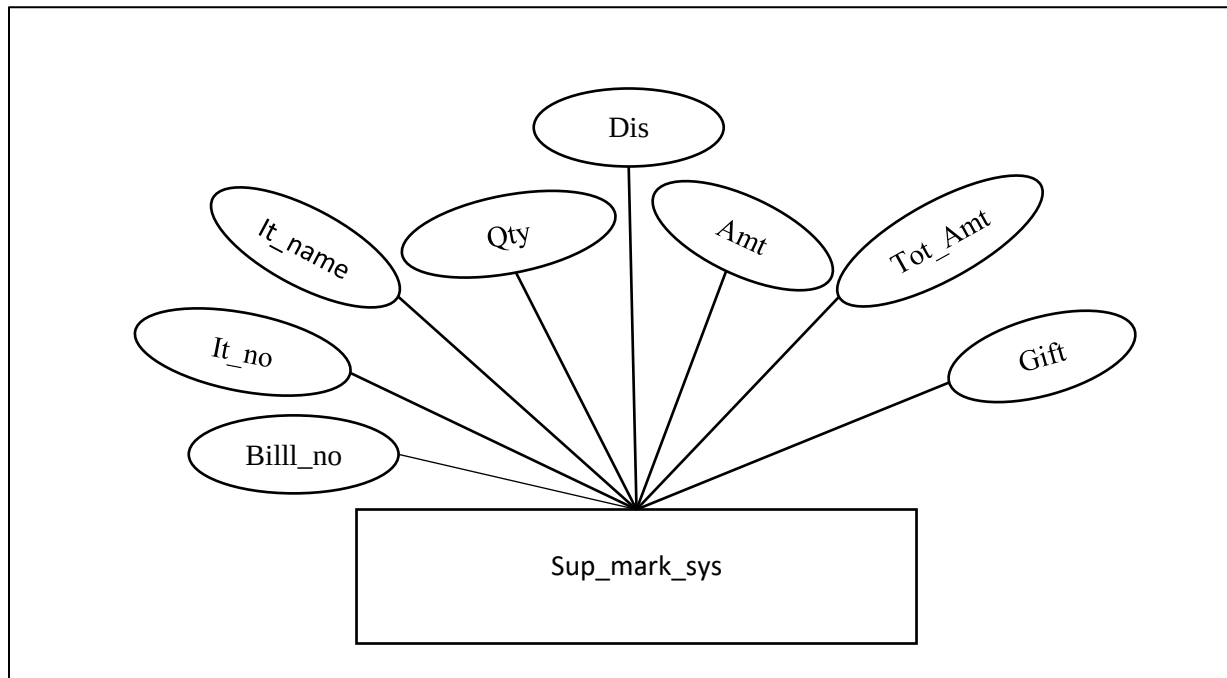
1 row created.

```
SQL> select * from sup_mark_sys;
```

BILL_NO	IT_NO	IT_NAME	QTY	DIS	AMT	TOT_AMT	GIFT
10	veg_30	Potato	30	0	100	100	N
09	veg_10	Onion	50	20	400	380	Y
11	frts_2	Papaya	5	20	300	280	Y
21	stat_4	Pen	5	0	50	30	N
15	stat_2	Note	20	10	600	590	Y
16	frts_5	Apple	35	30	1000	970	Y

6 rows selected.

E-R Diagram



Result:

Thus, the supermarket table was created and executed successfully.

Ex.No: 5	Computer Goods table for Purchase and Sales to perform relational algebra

Aim

To apply basic relational algebra operations — **Selection, Projection, Union, Set Difference,** and **Rename** — on computer-related goods data from Goods, Purchases, and Sales tables to analyze their purchase and sales activities.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Select Computer Goods
- ❖ **Step 3:** Select Specific Columns
- ❖ **Step 4:** Combine Purchases and Sales
- ❖ **Step 5:** Find Goods Purchased But Not Sold
- ❖ **Step 6:** Rename Columns
- ❖ **Step 7:** Stop

The following table expressed about attributes for Good, Purchase and Sales

Table : Goods

Attribute	Data Type	Description
GoodsID	INT	Unique ID for each product.
GoodsName	VARCHAR(100)	Name of the product
Category	VARCHAR(100)	Category the product belongs to

Table : Purchases

Attribute	Data Type	Description
PurchaseID	INT	Unique ID for each purchase.
GoodsID	INT	ID of the product purchased
Quantity	INT	Quantity of the product purchased.
Price	INT	Price per unit of the product.

Table : Sales

Attribute	Data Type	Description
SaleID	INT	Unique ID for each sale.
GoodsID	INT	ID of the product sold
Quantity	INT	Quantity of the product sold.
Price	DECIMAL(10, 2)	Price per unit of the product sold.

Structured Query Language (SQL)

```
SQL> CREATE TABLE Goods ( GoodsID INT PRIMARY KEY,  
3   GoodsName VARCHAR(100),  
4   Category VARCHAR(100) );
```

Table created.

```
SQL> CREATE TABLE Purchases (  
2   PurchaseID INT PRIMARY KEY,  
3   GoodsID INT,  
4   Quantity INT,  
5   Price DECIMAL(10, 2),  
6   FOREIGN KEY (GoodsID) REFERENCES Goods(GoodsID) );
```

Table created.

```
SQL> CREATE TABLE Sales (  
2   SaleID INT PRIMARY KEY,  
3   GoodsID INT,  
4   Quantity INT,  
5   Price DECIMAL(10, 2),  
6   FOREIGN KEY (GoodsID) REFERENCES Goods(GoodsID) );
```

Table created.

```
SQL> INSERT INTO Goods (GoodsID, GoodsName, Category) VALUES (1, 'Laptop', 'Electronics');  
1 row created.
```

```
SQL> INSERT INTO Goods (GoodsID, GoodsName, Category) VALUES (2, 'Smartphone', 'Electronics');  
1 row created.
```

```
SQL> INSERT INTO Goods (GoodsID, GoodsName, Category) VALUES (3, 'Desk Chair', 'Furniture');  
1 row created.
```

```
SQL> INSERT INTO Goods (GoodsID, GoodsName, Category) VALUES (4, 'Notebook', 'Stationery');  
1 row created.
```

```
SQL> INSERT INTO Goods (GoodsID, GoodsName, Category) VALUES (5, 'Water Bottle',  
'Kitchenware');  
1 row created.
```

```
SQL> INSERT INTO Purchases (PurchaseID, GoodsID, Quantity, Price) VALUES (101, 1, 10, 800.00);  
1 row created.
```

```
SQL> INSERT INTO Purchases (PurchaseID, GoodsID, Quantity, Price) VALUES (102, 2, 20, 400.00);  
1 row created.
```

```
SQL> INSERT INTO Purchases (PurchaseID, GoodsID, Quantity, Price) VALUES (103, 3, 15, 70.00);  
1 row created.
```

```
SQL> INSERT INTO Purchases (PurchaseID, GoodsID, Quantity, Price) VALUES (104, 4, 50, 2.50);  
1 row created.
```

```
SQL> INSERT INTO Purchases (PurchaseID, GoodsID, Quantity, Price) VALUES (105, 5, 30, 5.00);  
1 row created.
```

SQL> INSERT INTO Sales (SaleID, GoodsID, Quantity, Price) VALUES (201, 1, 7, 1000.00);
1 row created.

SQL> INSERT INTO Sales (SaleID, GoodsID, Quantity, Price) VALUES (202, 2, 18, 500.00);
1 row created.

SQL> INSERT INTO Sales (SaleID, GoodsID, Quantity, Price) VALUES (203, 3, 10, 120.00);
1 row created.

SQL> INSERT INTO Sales (SaleID, GoodsID, Quantity, Price) VALUES (204, 4, 40, 5.00);
1 row created.

SQL> INSERT INTO Sales (SaleID, GoodsID, Quantity, Price) VALUES (205, 5, 25, 10.00);
1 row created.

SQL> select * from Goods;

GOODSID	GOODSNAME	CATEGORY
1	Laptop	Electronics
2	Smartphone	Electronics
3	Desk Chair	Furniture
4	Notebook	Stationery
5	Water Bottle	Kitchenware

SQL> select * from Purchases;

PURCHASEID	GOODSID	QUANTITY	PRICE
101	1	10	800
102	2	20	400
103	3	15	70
104	4	50	2.5
105	5	30	5

SQL> select * from Sales;

SALEID	GOODSID	QUANTITY	PRICE
201	1	7	1000
202	2	18	500
203	3	10	120
204	4	40	5
205	5	25	10

SQL> SELECT DISTINCT GoodsID

2 FROM Purchases

3 INTERSECT

4 SELECT DISTINCT GoodsID

5 FROM Sales;

GOODSID

1
2
3
4
5

SQL> SELECT GoodsID, SUM(Quantity) AS TotalPurchased FROM Purchases GROUP BY GoodsID;
GOODSID TOTALPURCHASED

1 10
2 20
3 15
4 50
5 30

SQL> SELECT GoodsID, SUM(Quantity) AS TotalSold FROM Sales GROUP BY GoodsID;
GOODSID TOTALSOLD

1 7
2 18
3 10
4 40
5 25

SQL> SELECT g.GoodsID, g.GoodsName, g.Category,
COALESCE(SUM(p.Quantity), 0) - COALESCE(SUM(s.Quantity), 0) AS CurrentStock FROM Goods g
LEFT JOIN Purchases p ON g.GoodsID = p.GoodsID
LEFT JOIN Sales s ON g.GoodsID = s.GoodsID
GROUP BY g.GoodsID, g.GoodsName, g.Category;

GOODSID	GOODSNAME	CATEGORY	CURRENTSTOCK
1	Laptop	Electronics	3
2	Smartphone	Electronics	2
3	Desk Chair	Furniture	5
4	Notebook	Stationery	10
5	Water Bottle	Kitchenware	5

```
SQL> SELECT * FROM Goods WHERE Category = 'Electronics';
```

GOODSID	GOODSNAME	CATEGORY
1	Laptop	Electronics
2	Smartphone	Electronics

```
SQL> SELECT GoodsName, Category  
2 FROM Goods;
```

GOODSNAME	CATEGORY
Laptop	Electronics
Smartphone	Electronics
Desk Chair	Furniture
Notebook	Stationery
Water Bottle	Kitchenware

```
SQL> SELECT GoodsID FROM Purchases UNION SELECT GoodsID FROM Sales;
```

GOODSID
1
2
3
4
5

```
SQL> SELECT GoodsName AS NewGoodsName, Category FROM Goods;
```

NEWGOODSNAME	CATEGORY
Laptop	Electronics
Smartphone	Electronics
Desk Chair	Furniture
Notebook	Stationery
Water Bottle	Kitchenware

Result:

Thus the above table was executed successfully.

Ex.No: 6	Bank Customer table with Primary Key and execute 1NF and 2NF

Aim

To apply 1NF and 2NF on a Bank Customer table using SQL to remove duplicate and repeating data.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Create the Raw Table (Unnormalized)
- ❖ **Step 3:** Analyze and Identify Violations of 1NF
- ❖ **Step 4:** Convert to 1NF (Atomic Values)
- ❖ **Step 5:** Insert Data into Normalized Tables (1NF & 2NF)
- ❖ **Step 6 :** Verify Your Data
- ❖ **Step 7 :** Stop

The following table expressed about attributes for

Column Name	Data Type	Description
CustomerID	NUMBER	Unique ID for each customer
CustomerName	VARCHAR2(100)	Full name of the customer
PhoneNumbers	VARCHAR2(200)	Comma-separated phone numbers
Accounts	VARCHAR2(200)	Comma-separated account info

Structured Query Language (SQL)

--This table is not normalized

```
SQL> CREATE TABLE BankCustomerRaw (
2  CustomerID  NUMBER,
3  CustomerName VARCHAR2(100),
```

```

4  PhoneNumbers VARCHAR2(200),
5  Accounts     VARCHAR2(200)
6 );

```

Table created.

-- Sample Insert (unnormalized data)

```
SQL> INSERT INTO BankCustomerRaw VALUES (101, 'Alice Brown', '123-456,789-012', '001-Savings,002-Loan');
```

1 row created.

```
SQL> INSERT INTO BankCustomerRaw VALUES (102, 'Bob Smith', '555-999', '003-Savings');
```

1 row created.

```
SQL> select * from BankCustomerRaw;
```

CUSTOMERID	CUSTOMERNAME	PHONENUMBERS	ACCOUNTS
101	Alice Brown	123-456,789-012	001-Savings,002-Loan
102	Bob Smith	555-999	003-Savings

```
SQL> COMMIT;
```

Commit complete.

--Normalize to 1NF – Atomic Data

```
SQL> CREATE TABLE Customer (
```

```

2  CustomerID   NUMBER PRIMARY KEY,
3  CustomerName VARCHAR2(100)
4 );

```

Table created.

```
SQL> CREATE TABLE CustomerPhone (
```

```

2  CustomerID   NUMBER,
3  PhoneNumber  VARCHAR2(20),
4  CONSTRAINT pk_customerphone PRIMARY KEY (CustomerID, PhoneNumber),
5  CONSTRAINT fk_cp_customer FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)

```

6);

Table created.

SQL> CREATE TABLE Account (

2 AccountNumber VARCHAR2(10) PRIMARY KEY,

3 AccountType VARCHAR2(20)

4);

Table created.

SQL> CREATE TABLE CustomerAccount (

2 CustomerID NUMBER,

3 AccountNumber VARCHAR2(10),

4 CONSTRAINT pk_customeraccount PRIMARY KEY (CustomerID, AccountNumber),

5 CONSTRAINT fk_ca_customer FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),

6 CONSTRAINT fk_ca_account FOREIGN KEY (AccountNumber) REFERENCES
Account(AccountNumber)

7);

Table created.

--Insert Sample Data (Normalized to 2NF)

-- Insert Customers

SQL> INSERT INTO Customer VALUES (101, 'Alice Brown');

1 row created.

SQL> INSERT INTO Customer VALUES (102, 'Bob Smith');

1 row created.

-- Insert Customer Phones

SQL> INSERT INTO CustomerPhone VALUES (101, '123-456');

1 row created.

SQL> INSERT INTO CustomerPhone VALUES (101, '789-012');

1 row created.

```
SQL> INSERT INTO CustomerPhone VALUES (102, '555-999');
```

```
1 row created.
```

```
-- Insert Accounts
```

```
SQL> INSERT INTO Account VALUES ('001', 'Savings');
```

```
1 row created.
```

```
SQL> INSERT INTO Account VALUES ('002', 'Loan');
```

```
1 row created.
```

```
SQL> INSERT INTO Account VALUES ('003', 'Savings');
```

```
1 row created.
```

```
-- Link Customers to Accounts
```

```
SQL> INSERT INTO CustomerAccount VALUES (101, '001');
```

```
1 row created.
```

```
SQL> INSERT INTO CustomerAccount VALUES (101, '002');
```

```
1 row created.
```

```
SQL> INSERT INTO CustomerAccount VALUES (102, '003');
```

```
1 row created.
```

```
SQL> COMMIT;
```

```
Commit complete.
```

```
--Check Data
```

```
SQL> SELECT * FROM Customer;
```

```
CUSTOMERID CUSTOMERNAME
```

```
-----
```

```
101 Alice Brown
```

```
102 Bob Smith
```

```
SQL> SELECT * FROM CustomerPhone;
```

CUSTOMERID PHONENUMBER

101 123-456

101 789-012

102 555-999

SQL> SELECT * FROM Account;

ACCOUNTNUM ACCOUNTTYPE

001 Savings

002 Loan

003 Savings

SQL> SELECT * FROM CustomerAccount;

CUSTOMERID ACCOUNTNUM

101 001

101 002

102 003

Result :

Thus, the BankCustomerRaw table was normalized by applying 1NF and 2NF. Repeating and duplicate data were removed, and the data was organized into separate tables.

Ex.No: 7	Salary table and apply Statistical functions

Aim

To create salary table and perform statistical operations.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Create the table for salary
- ❖ **Step 3:** The attributes are EMP_ID,PROJECT and SALARY
- ❖ **Step 4:** Execute the queries for given table
- ❖ **Step 5:** Stop

The following table expressed about attributes for salary

Attributes	Description	Datatype
Emp_id	Employee id	Int
Project	Allocated Project	Varchar
salary	Salary	int

Structured Query Language (SQL)

```
SQL> CREATE TABLE SALARY ( EMP_ID INT NOT NULL,
PROJECT VARCHAR(50) NOT NULL,
SALARY INT);
```

Table created.

```
SQL> set lines 100;
```

```
SQL> desc salary;
```


Name	Null?	Type

EMP_ID	NOT NULL	NUMBER(38)
PROJECT	NOT NULL	VARCHAR2(50)
SALARY		NUMBER(38)

SQL> INSERT INTO SALARY VALUES (100,'P',20000);

1 row created.

SQL> INSERT INTO SALARY VALUES (101,'P2',40000);

1 row created.

SQL> INSERT INTO SALARY VALUES (102,'P3',50000);

1 row created.

SQL> INSERT INTO SALARY VALUES (103,'P3',50000);

1 row created.

SQL> INSERT INTO SALARY VALUES (104,'P2',40000);

1 row created.

SQL> INSERT INTO SALARY VALUES (105,'P4',15000);

1 row created.

SQL> INSERT INTO SALARY VALUES (106,'P1',20000);

1 row created.

SQL> INSERT INTO SALARY VALUES (107,'P5',70000);

1 row created.

SQL> INSERT INTO SALARY VALUES (108,'P5',55000);

1 row created.

SQL> select * from salary;

EMP_ID	PROJECT	SALARY
100	P	20000
101	P2	40000
102	P3	50000
103	P3	50000
104	P2	40000
105	P4	15000
106	P1	20000
107	P5	70000
108	P5	55000

9 rows selected.

Statistical functions

AVG()

SQL> select avg(salary) from salary;

AVG(SALARY)

40000

Sum()

SQL> select sum(salary) from salary;

SUM(SALARY)

360000

Count()

SQL> select count(emp_id) from salary;

COUNT(EMP_ID)

9

Max()

SQL> select max(salary) from salary;

MAX(SALARY)

70000

Min()

SQL> select min(salary) from salary;

MIN(SALARY)

15000

Variance ()

SQL> SELECT VARIANCE(salary) from salary;

VARIANCE(SALARY)

343750000

Standard Deviation()

SQL> SELECT STDDEV(salary) from salary;

STDDEV(SALARY)

18540.4962

Result

Thus, the salary table was created and executed successfully.

Ex.No: 8	Explore about single row function – Date functions

Aim

To apply date functions on dual table (Pre-Defined Table) using SQLPlus.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Pre-defined table is used which is named as “dual” in SQLPlus.
- ❖ **Step 3:** Execute the queries for given table
- ❖ **Step 4:** Stop

Structured Query Language (SQL)

SQL> select sysdate from dual;

SYSDATE

01-MAY-24

SQL> select sysdate+10 from dual;

SYSDATE+1

11-MAY-24

SQL> select sysdate-10 from dual;

SYSDATE-1

21-APR-24

SQL> SELECT TO_CHAR(SYSDATE, 'MM-DD-YYYY HH:MI:SS') "NOW" FROM DUAL;

NOW

05-01-2024 11:38:46

SQL> SELECT ADD_MONTHS(SYSDATE, 2) FROM DUAL;

ADD_MONTH

01-JUL-24

SQL> SELECT ADD_MONTHS(SYSDATE, -2) FROM DUAL;

ADD_MONTH

01-MAR-24

SQL> SELECT LAST_DAY(SYSDATE) FROM DUAL;

LAST_DAY(

31-MAY-24

SQL> SELECT SYSDATE, LAST_DAY(SYSDATE) "Last", LAST_DAY(SYSDATE) - SYSDATE "Days Left" FROM DUAL;

SYSDATE	Last	DaysLeft
----------------	-------------	-----------------

1-May-24	31-May-24	30
----------	-----------	----

SQL> SELECT NEXT_DAY(SYSDATE,'MONDAY') FROM DUAL;

NEXT_DAY(

06-MAY-24

SQL> SELECT NEXT_DAY(SYSDATE,'FRIDAY') FROM DUAL;

NEXT_DAY(

03-MAY-24

SQL> SELECT MONTHS_BETWEEN('05-JAN-2024','05-May-2023') FROM DUAL;

MONTHS_BETWEEN('05-JAN-2024','05-MAY-2023')

8

Result

Thus, the date functions were executed successfully.

Ex.No: 9	Explore about single row function – String functions

Aim

To apply string functions on dual table (Pre-Defined Table) using SQLPlus

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Pre-defined table is used which is named as “dual” in SQLPlus.
- ❖ **Step 3:** Execute the queries for given table
- ❖ **Step 4:** Stop

Structured Query Language (SQL)

SQL> SELECT ascii('B') from dual;

ASCII('B')

66

SQL> SELECT ascii('b') from dual;

ASCII('B')

98

SQL> SELECT LENGTH('BCA') FROM DUAL;

LENGTH('BCA')

3

SQL> SELECT LENGTH('B C A') FROM DUAL;

LENGTH('BCA')

5

SQL> SELECT LOWER('COMPUTER APPLICATIONS') FROM DUAL;

LOWER('COMPUTERAPPLIC

computer applications

SQL> SELECT upper('cyber security') FROM DUAL;

UPPER('CYBERSE

CYBER SECURITY

SQL> SELECT INITCAP('data base management system') FROM DUAL;

INITCAP('DATABASEMANAGEMENT

Data Base Management System

SQL> SELECT LTRIM('+++Good Luck ###','+') FROM DUAL;

LTRIM('+++GOO

Good Luck ###

SQL> SELECT rTRIM('+++Good Luck###','#') FROM DUAL;

RTRIM('+++GO

+++Good Luck

SQL> SELECT CONCAT('Multimedia', 'Systems') ConcatString FROM DUAL;

CONCATSTRING

MultimediaSystems

SQL> SELECT REPLACE('JACK and JUE','J','BL') "New String" FROM DUAL;

New String

BLACK and BLUE

```
SQL> SELECT TRANSLATE('HITS|UG-BCA|PG-MCA', '|', ',') AS "HITS-Cources" FROM DUAL;
```

HITS-Cources

HITS,UG-BCA,PG-MCA

```
SQL> SELECT SUBSTR('Animation Lab',1,3) FROM DUAL;
```

SUB

Ani

```
SQL> SELECT INSTR('Robotics-World','W') FROM DUAL;
```

INSTR('ROBOTICS-WORLD','W')

10

Result

Thus, the string functions were executed successfully.

Ex.No: 10	Display a hash value using default hash algorithm

Aim

To display the hash value of input data using Oracle's default hash function.

Procedure

- ❖ **Step 1:** Start SQL Plus
- ❖ **Step 2:** Accept inputs for the attributes such as pr_no, pr_name, pr_section and price
- ❖ **Step 3:** Insert the input values into the product table.
- ❖ **Step 4:** Commit the transaction to save the data permanently in the database.
- ❖ **Step 5:** Display the inserted product details using Select command.
- ❖ **Step 6:** Stop

The following table expressed about attributes for product

Attribute Name	Data Type	Description
pr_no	INT	Product Number
pr_name	CHAR(20)	Name of the Product
pr_section	CHAR(30)	Section or Category of Product
price	INT	Price of the Product in currency

Structured Query Language (SQL)

SQL> create table product (pr_no int,pr_name char(20),pr_section char(30),price int);
Table created.

SQL> insert into product values(101,'TV','Electronics',20000);
1 row created.

SQL> insert into product values (102,'Tomato','Vegetables ',200);
1 row created.

SQL> insert into product values (103,'Pen','Stationary',100);
1 row created.

SQL> insert into product values(115,'Apple','Fruits',450);
1 row created.

SQL> insert into product values(15,'Banana','Fruits',80);
1 row created.

SQL> select * from product;

PR_NO	PR_NAME	PR_SECTION	PRICE
101	TV	Electronics	20000
102	Tomato	Vegetables	200
103	Pen	Stationary	100

115	Apple	Fruits	450
15	Banana	Fruits	80

5 rows selected.

```
SQL> SELECT pr_no,STANDARD_HASH(pr_no,'MD5') FROM product;
PR_NO STANDARD_HASH(PR_NO,'MD5')
```

```
-----
102      A0A8A3F2F52FCDCCA15B036BFABAD618
103      440014D58E57E896830A69324E91C781
115      74ED7A887F229699FD68BDF4ECFF521B
15       35B5B4EA1D914B11747C54B4EDAACFF7
101      40F504E2830135189DCB9EDDAAED58C1
```

```
SQL> SELECT pr_name,STANDARD_HASH(pr_name,'MD5') FROM product;
PR_NAME      STANDARD_HASH(PR_NAME,'MD5')
```

```
-----
Tomato       9C87894362AEB69A7613A8301E47C773
Pen          DBF6A5EDEF5240A2162370CCD1580790
Apple        06A4F13403CC9B0EA6B11C574652060D
Banana       D7AA60F5650FFD4397E5A9A5F7335A23
TV           B7D7796E67B71F99A93BD4AEAD0CC95F
```

```
SQL> SELECT pr_section,STANDARD_HASH(pr_section,'MD5') FROM product;
PR_SECTION    STANDARD_HASH(PR_SECTION,'MD5')
```

```
-----
Vegetables    AA51F34090F5E9F980EFF9DBE5402225
Stationary    11BCA8D7E96C3EF70D58208F487AE4E9
Fruits        1F084065FC6C2B43321A92E9B56C7862
Fruits        1F084065FC6C2B43321A92E9B56C7862
Electronics   530E6FDB21D3B4C74D8C868E16A50BFD
```

```
SQL> SELECT price,STANDARD_HASH(price,'MD5') FROM product;
PRICE STANDARD_HASH(PRICE,'MD5')
```

```
-----
200      5C26A43DEE74C2B0005EC7C390083548
100      E6A66A1E449173C03BD767ACC48EE611
450      FE9ADF989D5E79552ED7A39183AF68AA
80       C92EE4B60FB4A64A556CE98040793B15
20000    C21F0C542FDFE2F4872940CAFEFA682B
```

Result:

Thus, the above table was created and executed successfully.